Selbstbalancierendes Fahrzeug

VMCB

Name:

Semester:

Datum: 15.06.2021

Verzeichnis

Abstract 4
1. Ziele 4
2. Vorstellung 4
2.1 Systementwurf 4
2.2 Systemanalyse 5
2.2.1 Steuermodul 5
2.2.2 Motorenantrieb 5
2.2.3 DC-Motoren mit Drehgeber 5
2.2.4 Beschleunigungssensor 6
2.2.5 Bluetooth-Modul 7
2.3 Algorithmus-Analyse 7
2.3.1 Verteilung der Steuerungsaufgaben 7
2.3.2 Winkelsteuerung 8
2.3.3 Geschwindigkeitssteuerung 10
2.4 Kenngrößen ermitteln 11
2.4.1 Neigungswinkel und Winkelgeschwindigkeit 11
2.4.2 Fahrgeschwindigkeit und gelaufene Strecke 11
2.4.3 Proportional- Integral- und Differential-Parameter 12
2.4.3.1 Parameter für die Winkelsteuerung 12
2.4.3.2 Parameter für die Geschwindigkeitssteuerung 13
2.5 Fernbedienen über Bluetooth 14
2.5.1 Kommunikation mit dem Bluetooth-Modul 14
2.5.2 App-Inventor 14
2.6 PCB über KiCad erstellen 15
2.6.1 Minimum-System vom STM32 15
2.6.1.1 Reset-Schaltung 15
2.6.1.2 ISP-Schaltung 16
2.6.1.3 Quarzoszillator- und Versorgungsschaltung 17
2.6.2 KiCad 17
2.6.2.1 Schaltplan mit EESchema erstellen 17

2.6.2.2 PCB-Layout erstellen 18
3. Zusammensetzen und testen 19
3.1 Zusammensetzen 19
3.2 Testen 19
3.3 Verbesserungsmöglichkeiten 20
Referenz: 21

Abstract

In den letzten Jahren haben die selbstbalancierenden Fahrzeuge wegen der Eigenschaften wie Flexibilität und Sparsamkeit bei Energie weit nach vorne entwickelt. Viele Forschungen beziehen sich auf dieses Thema und es gibt schon kommerzielle Produkte. Ein typisches Beispiel ist der Segway. Man sieht oft, dass die Fahrzeuge von Segway in den Städten herumlaufen.

In diesem Projekt wird es versucht, ein Segway-ähnliches Modell (experimental) zu erstellen. Ein Microcontroller dient als Hauptkontroller und mit Hilfe eines Beschleunigungssensors und zwei Drehgeber werden die Haltung und die Geschwindigkeit des Fahrzeugs ermittelt. Anhand der Messdaten wird ein PID-Algorithmus eingesetzt und damit versucht man, das Fahrzeug in Balance zu halten. Weiter kann man über Bluetooth mit einem Bedienteil (Smartphone) das Fahrzeug fernbedienen.

Schlüsselworte: PID-Algorithmus, Beschleunigungssensor, Drehgeber, geschlossener Regelkreis, Bluetooth

1. Ziele

Das Projekt hat folgende Ziele:

--die Lage des Fahrzeugs mit einem Beschleunigungssensor ermitteln (Winkel, Winkelbeschleunigung)

--die Motordrehzahl mit einem Drehgeber erfassen (Geschwindigkeit und Wegstrecke)

--mit den obigen Messdaten einen PID-Algorithmus einsetzen

--alle Bauteile auf eine per KiCad erstellte Platine löten und alle Komponenten zusammen an einem vorgefertigten Rahmen montieren

--mit dem PID-Algorithmus das Fahrzeug in Balance halten

--über Bluetooth das Fahrzeug fernbedienen

2. Vorstellung

2.1 Systementwurf

Anhand der Ziele und der Forderungen des Projekts ist ein Systementwurf in der Abbildung 2.1.1 gezeigt. Das Steuermodul ist die Kernkomponente des Systems. Der Beschleunigungssensor ermittelt den aktuellen Neigungswinkel und die Winkelbeschleunigung des Fahrzeugs und sendet zum Steuermodul die Messdaten. Die Drehgeber von den beiden Motoren senden die Drehimpulse zum Steuermodul und daraus wird die Motordrehzahl ermittelt. Das Steuermodul generiert geeignete PWM-Signale und diese werden zuerst durch den Motorantrieb verstärkt. Danach werden die Motoren mit den verstärkten Signalen betrieben. Das Bluetooth-Modul dient als Kommunikationsmittel zwischen dem Steuermodul und dem Bedienteil (Smartphone). Dadurch kann man Steuer-Signale zum Steuermodul schicken.

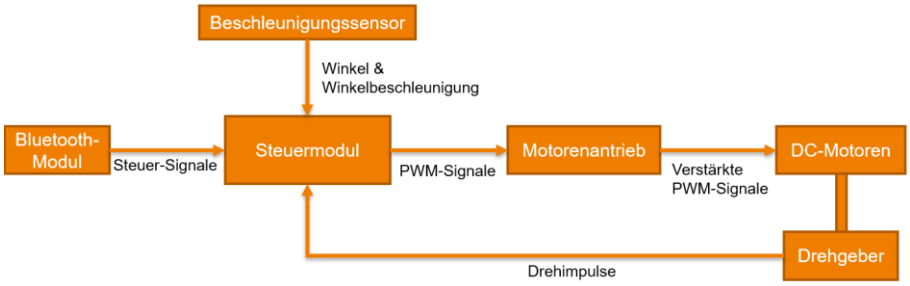

Abbildung 2.1.1 Blockschaltbild des ganzen Systems

2.2 Systemanalyse

2.2.1 Steuermodul

Der Mikrokontroller STM32F103RCT6 (STMicroelectronics) wurde als Hauptkontroller eingesetzt. Das ist ein 32-bit ARM Cortex-M3 Prozessor mit ausreichender Leistung für das Projekt. Das Steuermodul ist ein Minimum-System des Mikrokontrollers inklusive Reset-Schaltung, ISP-Schaltung, Quarzoszillator und Stromversorgung. Die ganze Schaltung wurde per KiCad als PCB erstellt.

2.2.2 Motorenantrieb

Der Motorenantrieb ist die Brücke zwischen dem Kontroller und dem Aktor. Die GPIOS des Mikrokontrollers können den Motor nicht direkt antreiben. Ein spezielles Motorenantriebmodul ist deswegen erforderlich. Hier wurde das Modell TB6612FNG(Toshiba) gewählt. Das ist ein Treiber IC für DC-Motoren mit Output Transistor in MOS Struktur. Im Vergleich zu dem üblichen Modell L298N besitzt es eine viel bessere Effizienz, deswegen braucht es keinen Kühlkörper und die Größe ist auch wesentlich kleiner. Das Modell TB6612FNG hat zwei Ausgänge mit Antriebsstrom von maximal 1,2A und die unterstützte PWM-Frequenz erreicht bis 100 kHz. Drinnen sind vier MOS-Transistoren gebaut und die bilden eine H-Brücke (Abbildung 2.2.2). Die vier Transistoren verteilen sich als zwei Gruppen, die abwechselnd an- und ausschalten. Der Motor kann damit in zwei Richtungen laufen. Die vier parallelschaltenden Dioden sind Schutzdioden, die die Transistoren vor Überspannung vom Motor schützen.

Abbildung 2.2.1 TB6612FNG Modul

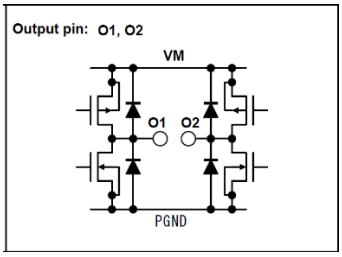

Abbildung 2.2.2 H-Brücke in TB6612FNG[1]

2.2.3 DC-Motoren mit Drehgeber

Es wurde zwei 12V-Gleichstrommotoren jeweils mit einem Drehzahlminderer eingesetzt. Die höchste Drehzahl beträgt 300 rpm. Zwei Drehgeber wurden jeweils an den beiden Motoren auf der gleichen Achse gebunden (Abbildung 2.2.3). Die Drehgeber sorgen für die Ermittlung der Motordrehzahl und der gelaufenen Strecke. Die sind optoelektronische Encoder mit einer Auflösung von 334PPR (Pulse Per Revolution). Das heißt, mit einem ganzen Kreislauf werden 334 Impulse ausgesendet.

Abbildung 2.2.3 DC-Motor mit Drehgeber

Drehgeber können in Bereich Lage- oder Winkelmessung eingesetzt werden. Die arbeiten alle auf Basis einer mechanischen

Bewegung (Rotation). Das grundlegende Messprinzip ist folgendes: eine Scheibe wurde auf der Drehachse festgebaut, dessen Rand mit einer bestimmten Anzahl von Teilstrichen gleichmäßig verteilt wurde. An einer Seite der Scheibe liegt eine LED, die direkt auf die Teilstrichen ausrichtet. An der anderen Seite wurde ein Fototransistor gebaut (Abbildung 2.2.4). Wenn sich der Motor in einem bestimmten Winkel dreht, strahlt das Licht von der LED durch die Teilstriche auf den Fototransistor und leitet er. Gleichzeitig wird ein Impuls ausgesandt. Nach einem vollständigen Kreis sendet der Ausgang bestimmte Anzahl von Impulsen aus. In einem bestimmten Zeitintervall wird die Impulse gezählt und man kann damit die Drehzahl ausrechnen. Je mehr Teilstriche ein Drehgeber besitzt, desto genauer ist die Auflösung.

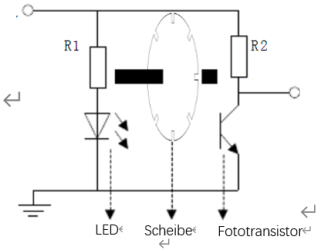

Abbildung 2.2.4[4]

2.2.4 Beschleunigungssensor

Als Beschleunigungssensor wurde hier das MPU6050-Modul eingesetzt. Das Modul ist ein 3-Achsen Beschleunigungssensor mit einem integrierten 3-Achsen Gyroskop. Das klassische Gyroskop (Abbildung 2.2.5) wird durch seine eigene Drehung stabilisiert und man muss eine zusätzliche Kraft darauf ausüben, um die Drehachse zu kippen. Diesen Effekt kann man sich zunutze machen, um die Lage eines Gegenstandes im Raum zu bestimmen. In einem MPU6050 wirkt das „Micro-Electric-Mechanical System“ (MEMS). Ein beweglicher aufgehängter Körper wird seine Lage bei Beschleunigung gegenüber einem festen Ort verändern. Durch diese Abstandsänderung ändert sich die Kapazität[6] (Abbildung 2.2.6).

Abbildung 2.2.5 Spielzeug Gyroskop[6]

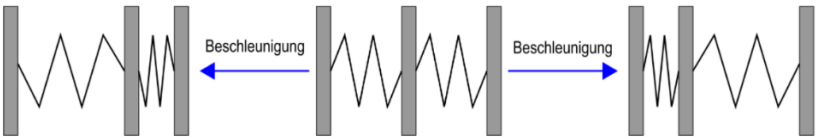

Abbildung 2.2.6 Spielzeug Gyrosko[6]

Nach demselben Prinzip funktioniert ein Beschleunigungssensor. Der Unterschied liegt nur daran, dass ein Beschleunigungssensor die Beschleunigung in Richtung der x-, y- und z-Achse detektiert und ein Gyroskop nur die Bewegung um die Drehachse detektiert. Ein Gyroskop liefert den Messwert Null, wenn es in Ruhe ist. Ein Beschleunigungssensor gibt aber nach der Gravitationsbeschleunigung bestimmte Messwerte in x-, y- und z-Richtung aus.

In diesem Modul wurde ein Mikrocontroller extra eingebaut und die originalen Messdaten vom MPU6050 werden von diesem Controller gesammelt und bearbeitet. Danach kann man die bearbeiteten Daten jeweils als Winkel, Winkelgeschwindigkeit und Winkelbeschleunigung über UART direkt ablesen. Die Messdaten von drei Achsen (Roll, Pitch und Yaw) stehen zur Verfügung. In diesem Projekt wird nur eine Achse gebraucht und die zu wählende Achse hängt von der Anbringungsstelle des Moduls ab.

Abbildung 2.8 MPU6050-Modul

2.2.5 Bluetooth-Modul

Bluetooth ist ein in den 1990er Jahren durch die Bluetooth Special Interest Group (SIG) entwickelter Industriestandard für die Datenübertragung zwischen Geräten über kurze Distanz per Funktechnik (WPAN). Dabei sind verbindungslose sowie verbindungsbehaftete Übertragungen von Punkt zu Punkt und Ad-hoc- oder Piconetze möglich. Der ursprüngliche Zweck war, die Datenübertragung von RS-232 durch eine verbindungslose Version zu ersetzen.

In diesem Projekt wurde das Bluetooth RF-Transceiver-Modul HC05 eingesetzt, um Kommunikation zwischen dem Fahrzeug und dem Bedienteil (Smartphone) aufzubauen. Das ist ein Master und Slave-zwei in einem Modul und hier arbeitet das Modul als Slave. Das Smartphone sendet als Master zum Bluetooth-Modul Steuersignale und das Modul kommuniziert mit dem Mikrocontroller über UART weiter. Dann kann der Mikrocontroller die gesendeten Steuerungssignale empfangen und die gewünschten Aktionen realisieren.

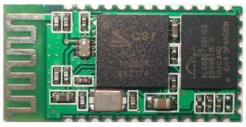

Abbildung 2.7 Bluetooth-Modul HC05

2.3 Algorithmus-Analyse

2.3.1 Verteilung der Steuerungsaufgaben

Ein selbstbalancierendes Fahrzeug hält sich im Gleichgewicht durch Vorwärts- oder Rückwärtsfahren der beiden Räder. Betrachtet man das Problem unter Steuerungsaspekt, dann hat das Fahrzeug als Steuerobjekt zwei Eingänge, nämlich die Drehzahlen der beiden Motoren. Die ganze Aufgabe kann in drei Teilaufgaben aufgeteilt werden:

1) Das Gleichgewicht zu halten: durch Steuerung der beiden Motoren
2) Die Fahrrichtung einzustellen: durch Einstellung der differentiellen Geschwindigkeit der beiden Motoren
3) Die Fahrgeschwindigkeit zu steuern: durch Einstellung des Neigungswinkels

Die Inhalte der ersten beiden Aufgaben sind direkte Steuerung von zwei Motoren. Angenommen der Motor kann in zwei virtuelle Motoren mit verschiedenen Funktionen zerlegt werden. Die beiden zerlegten Motoren sind auf der gleichen Achse verbunden und jeweils für das Stehenbleiben und Fahrrichtungsänderung des Fahrzeugs zuständig. Die Wirkungen der beiden Steuerungen überlappen sich. Die Fahrgeschwindigkeit wird durch Einstellung des Neigungswinkels geändert. Verschiedene Neigungswinkel entsprechen unterschiedlichen Beschleunigungen des Motors.

Die o.g. drei Teilaufgaben wirken selbstständig. Da das Steuerobjekt für die Teilaufgaben alle gleich ist (die beiden Motoren), besteht es Kopplungen zwischen den Teilaufgaben. Bei der Analyse einzelner Teilaufgabe soll man so annehmen, dass die zwei anderen Teilaufgaben schon erledigt sind und somit keinen Einfluss auf die aktuelle Teilaufgabe haben. Zum Beispiel bei der Fahrgeschwindigkeitsänderung soll das Fahrzeug schon zuvor erfolgreich selbst in Gleichgewicht halten können oder bei der Fahrrichtungsänderung soll das Fahrzeug schon kein Problem mit dem Gleichgewicht-halten und der Geschwindigkeitssteuerung haben.

Das Gleichgewicht zu halten ist der Angelpunkt der ganzen Aufgabe. Da das Fahrzeug von den drei Teilsteuerungen gleichzeitig beeinflusst wird, wirken die Teilsteuerungen als gegenseitige Störungen. Deswegen sollen die Fahrgeschwindigkeits- und Fahrrichtungssteuerung möglichst glatt ausgeführt werden, um den negativen Einfluss auf das Gleichgewicht-halten zu verringern.

2.3.2 Winkelsteuerung

Um das Gleichgewicht des Fahrzeugs zu halten, braucht man in der Steuerung eine negative Rückkopplung. Das Fahrzeug kann sich nur in der Rollrichtung der Räder neigen. Deswegen kann das Fahrzeug im Gleichgewicht halten, wenn die Räder das Fallen des Körpers verhindern können (Abbildung 2.8).

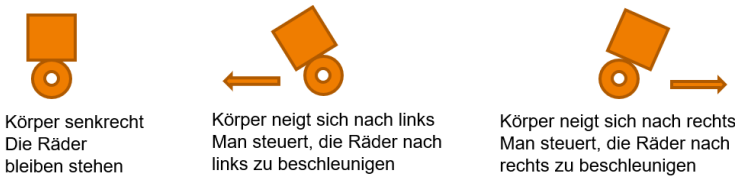

Abbildung 2.8 Negative Rückkopplung

Jetzt wird das dynamische Modell des Fahrzeugs gebildet, um einen Algorithmus zu entwickeln. Das Fahrzeug kann als ein inverses Fadenpendel mit einer beweglichen Plattform betrachtet werden (Abbildung 2.9).

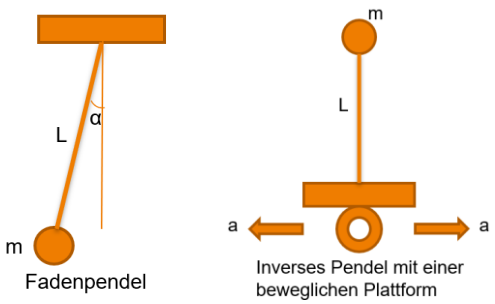

Abbildung 2.9 das Fahrzeug gilt als ein inverses Fadenpendel

Eine Kräfteanalyse des Fadenpendels steht in der Abbildung 2.10. Wenn die Kugel die Gleichgewichtslage verlässt, wirkt die resultierende Kraft von der Schwerkraft und der Fadenzugkraft auf die Kugel ein. Die Kugel wird dadurch gezwungen, wieder auf die Gleichgewichtslage zu gelangen. Die Größe der resultierenden Kraft beträgt (das Rundungszeichen wirksam bei sehr kleinem α)

$$F = mg\sin\alpha \approx mg\alpha \qquad (2\text{-}1)$$

Mit der Wirkung der resultierenden Kraft führt die Kugel eine Pendelbewegung aus. Die Dämpfungskraft, die durch die Reibung

an der das Pendel umgebenden Luft entsteht, bremst die Pendelbewegung und am Ende hält die Kugel wieder an der Gleichgewichtslage. Die Dämpfungskraft ist proportional zu der Kugelgeschwindigkeit, hat aber eine umgekehrte Richtung. Je größer die Dämpfungskraft ist, desto schneller hält die Kugel an.

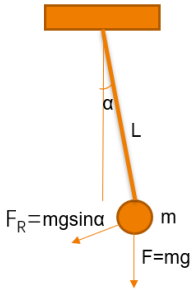

Abbildung 2.10 Kräfteanalyse eines einfachen Fadenpendels

Zusammenfassend lässt sich sagen, es sind zwei Voraussetzungen für das Anhalten der Kugel:

1) die resultierende Kraft hat eine umgekehrte Richtung als die Verschiebung

2) die Dämpfungskraft hat eine umgekehrte Richtung als die Kugelgeschwindigkeit

Ein inverses Pendel kann im Falle einer externen Störung nicht an der Vertikallage anhalten. Eine Kräfteanalyse steht in der Abbildung 2.11.

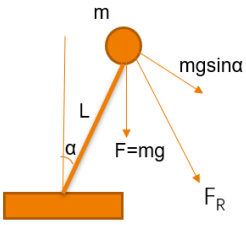

Abbildung 2.11 Kräfteanalyse eines inversen Pendels

Der Grund dafür, dass das inverse Pendel nicht an der Vertikallage anhalten kann, liegt an die Richtung der resultierenden Kraft. Wie die Abbildung 2.11 gezeigt, besitzt die resultierende Kraft F_R die gleiche Richtung wie die Verschiebung der Kugel und die Kugel wird sich somit immer beschleunigen, bis sie zu Boden fällt. Um das zu vermeiden, hat man zwei Möglichkeiten: Eine ist die Richtung der Schwerkraft zu ändern. Die andere ist eine zusätzliche Kraft auszuüben, damit die resultierende Kraft eine entgegengesetzte Richtung als die Verschiebung hat. Offensichtlich ist nur die zweite Möglichkeit machbar.

Jetzt steuert man das inverse Pendel so, dass sich der Unterteil nach der fallenden Richtung beschleunigt. Dann gilt das inverse Pendel als ein Nicht-Inertialsystem und eine Trägheitskraft übt auf das Pendel aus. Die Trägheitskraft hat eine umgekehrte Richtung als die Beschleunigung und ist proportional zu ihr (Abbildung 2.12).

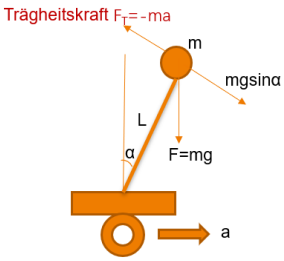

Abbildung 2.12 Kräfteanalyse eines beschleunigten inversen Pendels

Bei sehr kleinem α hat man in der x-Richtung die resultierende Kraft:

$$F_R = mg\sin\alpha - ma \approx mg\alpha - mk_1\alpha \quad (2\text{-}2)$$

In der Formel 2-2 ist die Beschleunigung a proportional zum Winkel α mit dem Faktor k_1. Solange k_1 größer als die Schwerebeschleunigung g ist, besitzt die resultierende Kraft eine entgegengesetzte Richtung als die Verschiebung. Um das Pendel möglichst schnell im Gleichgewicht zu halten, muss man eine Dämpfungskraft zuführen. Die natürliche Dämpfungskraft, die aus dem Luftwiderstand und der mechanischen Reibung stammt, reicht aber in diesem Fall nicht. Deswegen ist eine zusätzliche Dämpfungskraft erforderlich, deren Größe proportional zu der Winkelgeschwindigkeit und deren Richtung umgekehrt wie die Winkelgeschwindigkeit sind. Somit bekommt man die Formel 2-3:

$$F_R = mg\alpha - mk_1\alpha - mk_2\alpha' \quad (2\text{-}3)$$

Mit der Formel 2-3 kann man jetzt das inverse Pendel als ein Fadenpendel betrachten und es kann an der Vertikallage (möglichst) stabil anhalten. Der einzusetzende Ansatz lautet nämlich:

$$k_1\alpha + k_2\alpha' \quad (2\text{-}4)$$

In der Formel 2-4 ist α die Neigungswinkel des Fahrzeugs und α' ist die Winkelgeschwindigkeit. Die Parameter k_1 und k_2 sind die jeweiligen Proportional- und Differentialanteile. Das ist also ein PD-Regler. Mit geeigneten Werten der Parameter kann sich das Fahrzeug einigermaßen wie ein Fadenpendel verhalten. Es kann für kurze Zeit im Gleichgewicht halten. Den Neigungswinkel und die Winkelgeschwindigkeit kann man mit Hilfe eines Beschleunigungssensors ermitteln. Das wird im Abschnitt 2.4 weiter diskutiert.

2.3.3 Geschwindigkeitssteuerung

Mit der o.g. Winkelsteuerung kann das Fahrzeug zeitweilig stehenbleiben. Wenn es aber später eine große Neigung hat, wird es sich immer nach der Richtung der Neigung beschleunigen und dann hinunterfallen. Deswegen reicht die Winkelsteuerung allein nicht, das Fahrzeug möglichst gut zu stabilisieren. Hier muss man noch eine Geschwindigkeitssteuerung einsetzen.

Die Idee ist, eine positive Rückkopplung mit der Geschwindigkeit der Räder zu bilden. Betrachtet man die Abbildung 2.13, das Fahrzeug neigt sich in einem Zeitpunkt nach rechts. Dann wirkt die Winkelsteuerung und das Fahrzeug fährt ebenfalls nach rechts. Diese Steuerung reicht aber nicht, das Neigen rechtzeitig zu stoppen (die Winkelsteuerung soll nicht zu stark eingesetzt werden, sonst wird das System instabil) und die Folge ist, das Fahrzeug beschleunigt sich weiter nach rechts und dann hinunterfällt. Um das zu vermeiden, müssen die Räder aktiv darin eingreifen. Wenn das Fahrzeug die Tendenz zu kippen hat, beschleunigen sich die Räder noch schneller nach der Kipprichtung, um die Tendenz auszugleichen. Das ist eine positive Rückkopplung.

Abbildung 2.13 Das Fallen durch Beschleunigen der Räder verhindern

Die aktuelle Geschwindigkeit der beiden Räder dient als die Eingangsvariable der Geschwindigkeitssteuerung. Um die Steuerung zu vergrößern, multipliziert man die noch mit dem Parameter k_3 (das Proportionalanteil). Ein weiterer Parameter k_4 (Integralanteil) hilft dabei, die Verschiebung des Fahrzeugs zu eliminieren. Das ist also ein PI-Regler und der einzusetzende Ansatz lautet:

$$k_3 * v + k_4 * s \qquad (2\text{-}5)$$

In der Formel 2-5 ist v die Geschwindigkeit der Räder und s ist die gelaufene Strecke. Die beiden Größen kann man mit einem Drehgeber ermitteln. Das wird im Abschnitt 2.4 weiter diskutiert.

Zusammenfassend lautet der Algorithmus:

$$k_1\alpha + k_2\alpha' + k_3 * v + k_4 * s \qquad (2\text{-}6)$$

Das Ergebnis der Formel 2-6 wird direkt dem Motor als PWM-Signal zugewiesen.

2.4 Messgrößen ermitteln

2.4.1 Neigungswinkel und Winkelgeschwindigkeit

Die Formel 2-6 erfordert insgesamt vier Messwerte. Zwei davon sind der Neigungswinkel und die Winkelgeschwindigkeit, die mittels dem Beschleunigungssensor MPU6050 festgestellt werden können. Die Messdaten werden über UART direkt ausgegeben und man muss nur den UART im Hauptkontroller richtig konfigurieren, um die Messdaten abzulesen.

Das Programmieren über das Empfangen der Messdaten muss sich am Frame der Messdaten anpassen. Die Messdaten werden mit verschiedenen Headers versehen, damit man die jeweiligen Daten für die 3-Achsen erkennen kann[7] (Abbildung 2.4.1).

```
01. if(sign)
02. {
03.     sign=0;
04.     if(Re_buf[0]==0x55) // Daten empfangen
05.     {
06.         switch(Re_buf [1])//Header
07.         {
08.             case 0x51://Header für Beschleunigung
09.                 a[0] = (short(Re_buf [3]<<8| Re_buf [2]))/32768.0*16;//x-Achse
10.                 a[1] = (short(Re_buf [5]<<8| Re_buf [4]))/32768.0*16;//y-Achse
11.                 a[2] = (short(Re_buf [7]<<8| Re_buf [6]))/32768.0*16;//z-Achse
12.                 T = (short(Re_buf [9]<<8| Re_buf [8]))/340.0+36.25;
13.             break;
14.             case 0x52://Header für Winkelgeschwindigkeit
15.                 w[0] = (short(Re_buf [3]<<8| Re_buf [2]))/32768.0*2000;//x-Achse
16.                 w[1] = (short(Re_buf [5]<<8| Re_buf [4]))/32768.0*2000;//y-Achse
17.                 w[2] = (short(Re_buf [7]<<8| Re_buf [6]))/32768.0*2000;//z-Achse
18.                 T = (short(Re_buf [9]<<8| Re_buf [8]))/340.0+36.25;
19.             break;
20.             case 0x53://Header für Winkel
21.                 angle[0] = (short(Re_buf [3]<<8| Re_buf [2]))/32768.0*180;//x-Achse
22.                 angle[1] = (short(Re_buf [5]<<8| Re_buf [4]))/32768.0*180;//y-Achse
23.                 angle[2] = (short(Re_buf [7]<<8| Re_buf [6]))/32768.0*180;//z-Achse
24.                 T = (short(Re_buf [9]<<8| Re_buf [8]))/340.0+36.25;
25.             break;
26.         }
27.     }
28. }
```

Abbildung 2.4.1 Code fürs Empfangen der Messdaten vom MPU6050

2.4.2 Fahrgeschwindigkeit und gelaufene Strecke

Die zwei weiter benötigten Messewerte in Formel 2-6 sind die Fahrgeschwindigkeit und die gelaufene Strecke. Die kann man mit

Hilfe eines Drehgebers ermitteln. Das Messprinzip eines Drehgebers wurde im Abschnitt 2.2.3 schon vorgestellt. In der Software muss man den Hauptkontroller so konfigurieren, dass die Signale von den Drehgebern empfangen und in Geschwindigkeit und Strecke umgerechnet werden können.

Bei dem STM32F103RCT6 können die Timer-Einheiten als Encoder-Modul arbeiten. In diesem Fall kann man die entsprechenden Pins als Inputs konfigurieren und die Pins direkt mit den Ausgängen des Drehgebers verbinden. Die vom Drehgeber gesendeten Impulse lösen den Encoder aus und es wird gezählt, wie viele Impulse empfangen werden. Setzt man noch einen anderen Timer-Interrupt ein, dann kann man in einem bestimmten Zeitintervall anhand der Anzahl der Impulse die Fahrgeschwindigkeit berechnen. Die gelaufene Stracke entspricht dem Zeitintegral der Fahrgeschwindigkeit. Das kann man im Timer-Interrupt die neu berechnete Fahrgeschwindigkeit jedes Mal direkt summieren.

```
01. void TIM4_IRQHandler(void)   //TIM4 Overflow Interrupt
02. {
03.  if (TIM_GetITStatus(TIM4, TIM_IT_Update) != RESET)
04.   {
05.    TIM_ClearITPendingBit(TIM4, TIM_IT_Update  ); //Interrupt-Flag zurücksetzen
06.    speed = (TIM2_Encoder_Read_1() + TIM3_Encoder_Read_2())/2;//die Encoder-Werte ablesen
07.    TIM2_Encoder_Write_1(0);   //Encoder wieder zurücksetzen
08.    TIM3_Encoder_Write_2(0);   //Encoder wieder zurücksetzen
09.    speed_filter *= 0.85;//Tiefpass, um Störung zu unterdrücken
10.    speed_filter += speed * 0.15; //die Fahrgeschwindigkeit
11.
12.    position += speed_filter;  //die Fahrgeschindigkeiten summieren und die Summe ist die Strecke
13.    if(position > 40000)//Begrenzung
14.    {
15.     position = 40000;
16.    }
17.    else if(position < -40000)
18.    {
19.     position = -40000;
20.    }
21.   }
22.
23. }
```

Abbildung 2.4.2 Ermittlung der Fahrgeschwindigkeit und der Strecke

2.4.3 Proportional- Integral- und Differential-Parameter

Man braucht in der Formel 2-6 vier PID-Parameter: Proportional- und Differential-Glied für die Winkelsteuerung und Proportional- und Integral-Glied für die Geschwindigkeitssteuerung. Die kann man jeweils als Kp_angle, Kd_angle, Kp_speed und Ki_speed benennen (Formel 2-7). Die Bestimmung der vier Parameter ist entscheidend. Es gibt dafür aber ein empirisches Verfahren.

$$Kp_angle * \alpha + Kd_angle * \alpha' + Kp_speed * v + Ki_speed * s \qquad (2\text{-}7)$$

Zuerst soll man die mechanische Struktur des Fahrzeugs beachten. Man baut es so auf, dass der Schwerpunkt möglichst in der Mitte steht und damit das Fahrzeug auch ohne elektrische Steuerung nicht schlagartig hinfallen wird.

2.4.3.1 Parameter für die Winkelsteuerung

Die Parameter für die Winkelsteuerung sind Kp_angle und Kd_angle und die entsprechenden Messwerte sind jeweils der Neigungswinkel und die Winkelgeschwindigkeit. Vom MPU6050-Modul bekommt man wie im Abschnitt 2.4.1 erklärt 3-Achsen Winkel: Pitch, Roll und Yaw. Welche Achse soll man wählen, hängt von der Anbringungsstelle des Moduls ab. In diesem Projekt wurde der Roll-Winkel gewählt.

Zuerst soll der Wertbereich von Kp_angle bestimmt werden. Die verwendete PWM-Einheit hat einen maximalen Timer-Wert von 900, der 100% Tastgrad entspricht. Mit diesem Wert läuft der Motor mit voller Geschwindigkeit. Die Körperschwingung des Fahrzeugs soll in 10 Grad begrenzt werden. Deswegen hat Kp_angle einen Wertbereich von circa 1 bis 90. Dann soll deren Polarität bestimmt werden. Da die Winkelsteuerung eine negative Rückkopplung ist, soll das Fahrzeug mit einer geeigneten Polarität so verhalten, dass die Räder das Fallen des Körpers verhindern. Wenn es dagegen das Fallen beschleunigt, stimmt die Polarität nicht. Die Größe des Kp_angle wird durch mehrmaliges Ausprobieren bestimmt. Man kann bei dem Wert Eins anfangen und jedes Mal den Wert leicht erhöhen, bis er groß genug ist und es heißt, das Fahrzeug hat eine niederfrequente Schwingung mit einer großen Amplitude. So werden die Größe und die Polarität von Kp_angle bestimmt.

Die Bestimmung von Kd_angle ist ähnlich wie beim Kp_angle. Man setzt Kp_angle vorübergehend wieder auf Null und stellt zuerst die Polarität des Kd_angle fest. Hier wirkt ebenfalls eine negative Rückkopplung. Eine geeignete Polarität soll bewirken, dass die Räder nach der gleichen Richtung drehen, wenn das Fahrzeug manuell um die Radachse gedreht wird. Es ist hier zu beachten, dass sich die Räder nur bewegen sollen, wenn das Fahrzeug ebenfalls in Bewegung gesetzt wird, weil die Ausgabe dieser Steuerung von der aktuellen Winkelgeschwindigkeit abhängt. Und je größer die Winkelgeschwindigkeit ist, desto größer wird die Ausgabe. Danach kann man die Größe des Kd_angle zusammen mit Kp_angle bestimmen. Ähnlich wie beim Kp_angle, man kann mit einem kleinen Wert anfangen und jedes Mal den Wert leicht erhöhen, bis eine hochfrequente Schwingung mit kleiner Amplitude auftritt. So werden die Größe und die Polarität von Kd_angle bestimmt.

Allein mit der Wirkung der Winkelsteuerung kann das Fahrzeug kurzzeitig (i.d.R. in wenigen Sekunden) im Gleichgewicht halten. Das Fahrzeug wird eine immer größere Neigung haben und am Ende hinfallen.

2.4.3.2 Parameter für die Geschwindigkeitssteuerung

Die Parameter für die Geschwindigkeitssteuerung sind Kp_speed und Ki_speed und die entsprechenden Messwerte sind jeweils die Fahrgeschwindigkeit und die gelaufene Strecke.

Zuerst bestimmt man die Polarität des Kp_speed. Man schaltet vorübergehend die Winkelsteuerung aus (Kp_angle und Kd_angle auf Null setzen) und weist dem Kp_speed einen willkürlichen Wert zu. Da die Geschwindigkeitssteuerung eine positive Rückkopplung ist, soll eine geeignete Polarität bewirken, dass sich die beiden Räder bis zum Maximum beschleunigen, wenn man eines davon manuell nach einer Richtung dreht. Dreht das Rad aber gegenläufig, wenn man das Andere manuell dreht, dann stimmt die Polarität nicht. Danach kann man die Größen von Kp_speed und Ki_speed zusammen durch Ausprobieren feststellen. Man fängt wieder mit einem kleinen Wert an. Erfahrungsgemäß sollte man den Effekt so beurteilen: wenn sich das Fahrzeug willkürlich nach einer Richtung langsam bewegt und nicht wieder zurückkommen kann, ist die Wirkung von Kp_speed und Ki_speed zu klein. Wenn das Fahrzeug aber immer hin und zurück schaukelt und beim Schaukeln eine relativ große Neigung hat, ist die Wirkung zu groß. Man wählt eine geeignete Kombination von den beiden Größen.

Die Bestimmung der obigen vier PID-Parameter ist erfahrungsmäßig, weil das Verhalten des Fahrzeugs von vielen verschiedenen Faktoren abhängt und ein exaktes mathematisches Modell lässt sich schwierig aufzubauen.

2.5 Fernbedienen über Bluetooth

2.5.1 Kommunikation mit dem Bluetooth-Modul

Das Bluetooth-Modul HC05 wurde in diesem Projekt eingesetzt. Es dient als Medium zwischen dem Mikrocontroller und dem Bedienteil (Android Smartphone). Die Charaktere des HC05-Moduls kann man über die AT-Commands konfigurieren. Es gibt dafür eine Befehle-Versammlung. Dort wird vorgeschrieben, wie man zum Beispiel die PIN, die Baudrate und die Rolle (Master/Slave) des Moduls über ein UART-Terminal ändern kann. Mit der geeigneten PIN kann die kabellose Verbindung zwischen dem Smartphone und dem HC05-Modul aufbauen und die beiden Geräte können miteinander kommunizieren.

Eine UART-Schnittstelle wurde auf dem HC05-Modul zur Verfügung gestellt und dadurch kommuniziert der Mikrocontroller mit dem Modul. Dafür muss man die geeignete GPIOs, die Baudrate und das Datenframe im Programm konfigurieren. Die vom Smartphone gesendeten Befehle sind die Vorwärts/Rückwärtsbewegung und die nach links/rechts Abbiegung und diese lassen sich über die Plattform „App-Inventor“ realisieren.

Abbildung 2.5.1 Hardware-Aufbau der Bluetooth-Kommunikation

2.5.2 App-Inventor

Der App-Inventor ist eine vom US-amerikanischen Unternehmen Google entwickelte Entwicklungsumgebung, um Anwendungen für Android zu programmieren. Per Drag and Drop können Anwender die grafischen Blöcke (grafische Programmiersprache) zusammenbauen und somit Applikationen für Mobiltelefone mit dem Android-System erstellen.

Zuerst soll man den Bluetooth-Client aufrufen, um eine Verbindung zwischen dem Smartphone und dem HC05-Modul zu erstellen. Dafür braucht man die Bluetooth-Adresse des Moduls. Das ist ein 48-Bit Identifikator und den kann man vom zugehörigen Datenblatt ablesen. Danach kann man die gewünschten Befehle (Datenframe) aufbauen und zusenden. Jedes Datenframe wurde mit einem Header versehen, damit das Programm im Mikrocontroller die entsprechenden Anweisungen erkennen kann. Es sind insgesamt vier Befehle, stehen für jeweils die vier Bewegungsrichtung. Ein Beispielcode steht in der Abbildung 2.5.2 und die Benutzerschnittstelle der Applikation sieht man in der Abbildung 2.5.3.

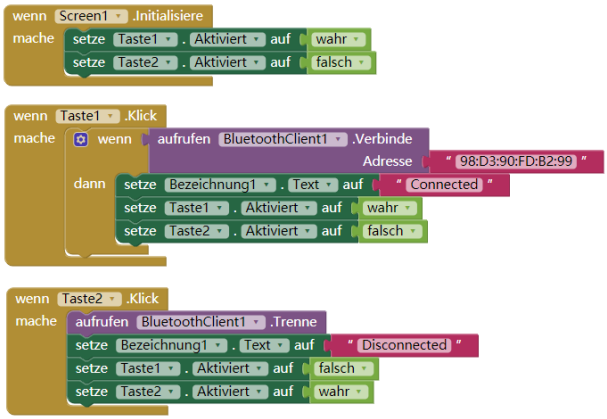

Abbildung 2.5.2 Beispielcode für die Bluetooth-Verbindung

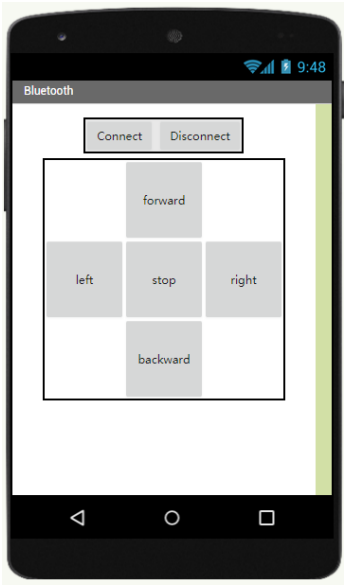

Abbildung 2.5.3 Benutzerschnittstelle der Applikation

2.6 PCB über KiCad erstellen

2.6.1 Minimum-System vom STM32

Hier soll ein Minimum-System vom Mikrocontroller STM32F103RCT6 aufgebaut werden. Das Minimum-System besteht aus dem Mikrocontroller, der Reset-Schaltung, der ISP-Schaltung, dem Quarzoszillator und der Stromversorgung.

2.6.1.1 Reset-Schaltung

Die Reset-Schaltung sendet einen Reset-Impuls beim Einschalten des Systems aus, damit sich der Mikrocontroller in einer gewissen Zeit im Reset-Zustand befindet. Erst nach dieser Zeit fängt der Mikrocontroller an zu arbeiten, um sicherzustellen, dass die Versorgungs- und die Oszillatorschaltung schon in stabilem Zustand sind. Es ist für die Stabilisierung des Systems entscheidend, eine geeignete Reset-Zeit zu wählen. Das lässt sich mit der folgenden Schaltung (Abbildung 2.6.1) realisieren.

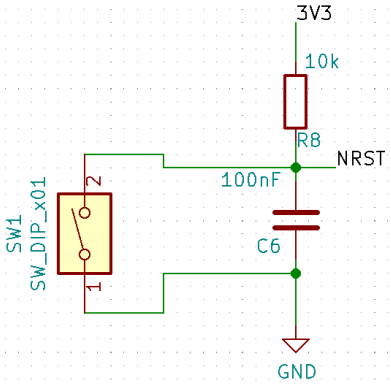

Abbildung 2.6.1 Reset-Schaltung

Der Reset-Eingang des STM32 (NRST) ist Low-aktiv. Beim Einschalten wird der Kondensator (C6) über den Widerstand (R8)

aufgeladen. Am Anfang existiert es keine Spannungsdifferenz zwischen den beiden Seiten des Kondensators und das PIN NRST bekommt einen Low-Pegel. In diesem Zeitpunkt hat der Strom den maximalen Wert. Mit dem Aufladen des Kondensators wird der Strom allmählich kleiner, bis die Spannungsdifferenz gleich VCC ist. Dann fließt es bei dem Kondensator keinen Strom mehr und das PIN NRST bekommt einen High-Pegel (VCC). Laut Datenblatt des STM32 beträgt die Reset-Zeit 1,5 bis 4,5ms. Die Zeitkonstante der RC-Schaltung beträgt $\tau = R * C = 10kOhm * 100nF = 1ms$. Eine vollständige Aufladung des Kondensators dauert 3 bis 5τ. Das heißt, der Low-Pegel am NRST dauert 3 bis 5ms. Die Länge erfüllt die oben genannte Reset-Zeit.

Der Taster SW1 sorgt für das manuelle Reset. Beim Bestätigen des Tasters bekommt das NRST sofort einen Low-Pegel und der Mikrocontroller wird somit zurückgesetzt.

2.6.1.2 ISP-Schaltung

Die ISP-Schaltung (In-System Programming) ermöglicht es Anwendern, das Flashen des Mikrocontrollers per UART durchzuführen. Dafür soll man noch klären, welche Boot-Mode es beim STM32 gibt. Mit verschiedenen Konfigurationen der zwei PINs (BOOT1 und BOOT0) kann man feststellen, wie und von welcher Adresse der Mikrocontroller starten wird. Genaue Beschreibung sieht man in der Tabelle 2.6.1.

Boot mode selection pins		Boot mode	Aliasing
BOOT1	BOOT0		
x	0	Main Flash memory	Main Flash memory is selected as boot space
0	1	System memory	System memory is selected as boot space
1	1	Embedded SRAM	Embedded SRAM is selected as boot space

Tabelle 2.6.1 Boot-Mode vom STM32 (Auszug vom Datenblatt)

Es sind insgesamt drei Boot-Mode:

1) BOOT1 = x, BOOT0 = 0: der Code im Flash wird durchgeführt
2) BOOT1 = 0, BOOT0 = 1: der Code von Anwendern wird per UART heruntergeladen und im Flash gespeichert
3) BOOT1 = 1, BOOT0 = 1: der Code im SRAM wird durchgeführt (für Debuggen)

Laut obiger Beschreibung soll man zuerst BOOT1 mit 0 und BOOT0 mit 1 konfigurieren. So wird der Code im Mikrocontroller heruntergeladen (Flashen). Danach soll der Mikrocontroller einmal zurückgesetzt werden. Und dann soll man BOOT0 wieder auf 0 setzen, damit der Code im Flash weiter durchgeführt werden kann. Um den Flashen-Vorgang zu realisieren, wurde folgende Schaltung eingesetzt:

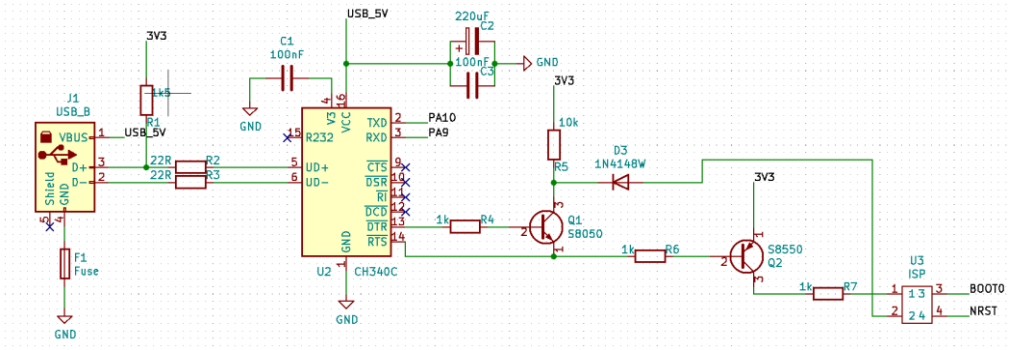

Abbildung 2.6.2 ISP-Schaltung

Der USB auf Seriell ISP-Chip CH340C ermöglicht eine Kommunikation zwischen dem Mikrocontroller und dem Host-Computer, damit die HEX-Datei auf dem Mikrocontroller heruntergeladen werden kann. Die PINs DTR und RTS steuern mit Hilfe der zwei Transistoren Q1 und Q2 auf die BOOT0 und NRST, um die Boot-Mode des Mikrocontrollers zu konfigurieren.

2.6.1.3 Quarzoszillator- und Versorgungsschaltung

Die Quarzoszillatorschaltung (Abbildung 2.6.3) arbeitet als Taktgeber und bietet für den Mikrocontroller eine Arbeitsfrequenz an.

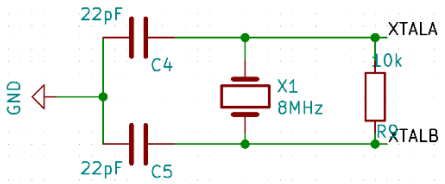

Abbildung 2.6.3 Quarzoszillatorschaltung

Das Motorantrieb-Modul, das MPU6050-Modul und das Bluetooth-Modul arbeiten unter einer Versorgungsspannung von 5V, deswegen wurde hier der Spannungsregler LM7805 eingesetzt. Mit einem 7,4V-Akkusatz als Input biete der LM7805 für das System eine stabile 5V an. Und es ist gleichzeitig der Input für den Spannungsregler LD1117, der den Mikrocontroller mit einer 3,3V versorgt.

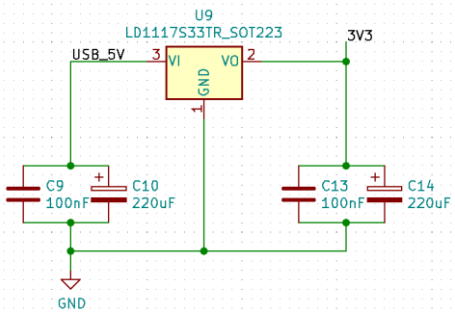

Abbildung 2.6.4 LD1117-Versorgungsschaltung

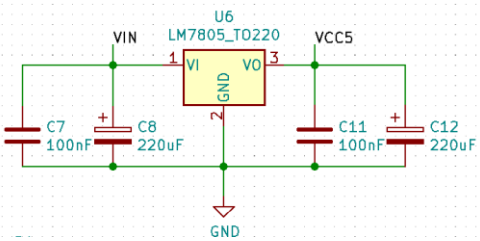

Abbildung 2.6.5 LM7805-Versorgungsschaltung

2.6.2 KiCad

KiCad ist ein freies ECAD-Programmpaket zur Entwicklung von Leiterplatten in der Elektronik. Mit KiCad kann man ein PCB-Layout entwickeln und die erzeugten Daten können nach einem PCB-Hersteller geschickt werden. Man bekommt damit das selbst entwickelte PCB.

2.6.2.1 Schaltplan mit EESchema erstellen

Zuerst wird mit EESchema ein Schaltplan unter Verwendung der Schaltplansymbole aus der Symbolbibliothek erstellt. Hier wird es

verdeutlicht, wie die einzelnen Bauteile miteinander verbunden sind. Das genaue Modell und die Parameter der Bauteile sollen auch angegeben werden. So hat man einen guten Überblick über den gesamten Schaltplan. Danach wird eine Netzliste erstellt. Hier werden alle verwendeten Bauteile aufgelistet und man muss jedem Bauteil ein Footprint zuweisen. Hier wird es festgelegt, mit welchem Gehäuse das Bauteil auf der Platine montiert wird. Es ist für die weitere Bearbeitung der PCB-Erstellung nötig. Der gesamte Schaltplan sieht man in der Abbildung 2.6.6.

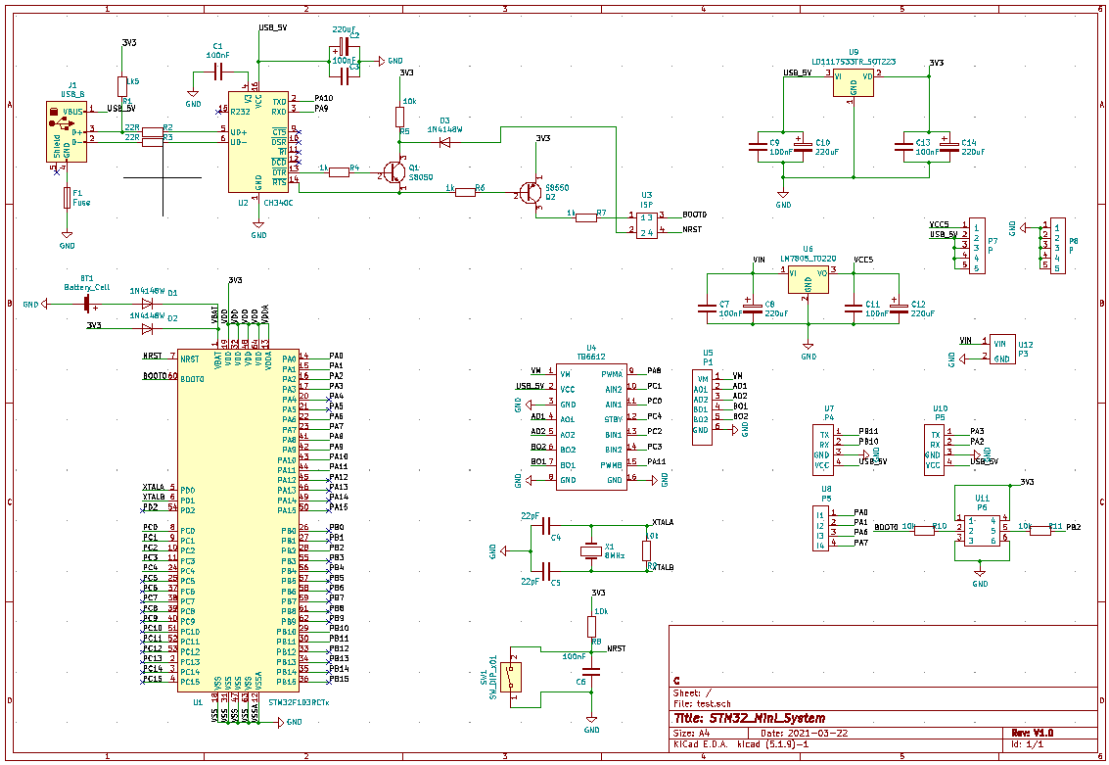

Abbildung 2.6.6 Gesamter Schaltplan

2.6.2.2 PCB-Layout erstellen

Danach werden die Bauteile auf der Platine platziert und mit Leiterbahnen verbunden. Hier kann man auch Netzklassen definieren, zum Beispiel die Leiterbahnbreiten und Isolationsabstände feststellen. Hier soll man einige Punkte beachten, zum Beispiel genug Platz für den USB-Stecker und die anderen PINs zu reservieren, die Kondensatoren möglichst in der Nähe vom Spannungsregler zu platzieren und deren Verbindungen möglichst kurz zu halten. Bei Hochfrequenzanwendungen gibt es besonders Regeln für die Anzahl der Bestückungslöcher, die Erdleitung und so weiter. Bei diesem Projekt kann man das aber sparen. Am Ende kann man die Layout-Datei herleiten und nach einem PCB-Hersteller schicken und dort produzieren lassen. Man bekommt die fertige Platine wie in der Abbildung 2.6.7.

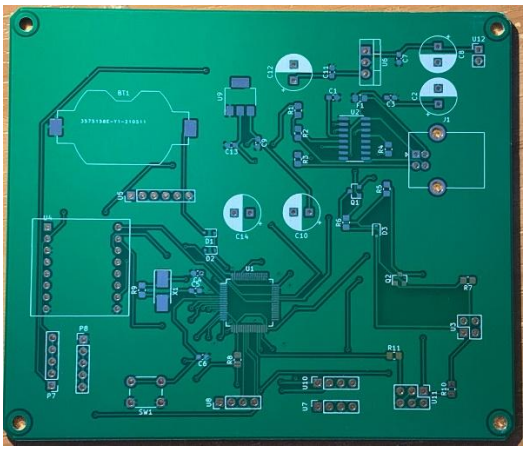

Abbildung 2.6.7 Fertiges PCB

3. Zusammensetzen und testen

3.1 Zusammensetzen

Nach Erhalt aller Bauteile kann die Platine gelötet und montiert werden. Zwei Akkusätze (7,4V und 11,1V) dienen jeweils als Versorgungen für das Steuer- und Motorantriebsmodul. An einem vorgefertigten Rahmen wurden alle Komponenten montiert. Man bekommt damit einen Prototyp wie in der Abbildung 3.1.1.

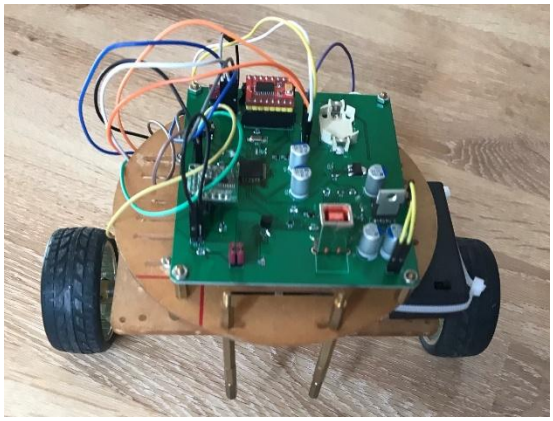

Abbildung 3.1.1 Prototyp

3.2 Testen

Die erste Aufgabe ist, das Fahrzeug im Gleichgewicht zu halten. Auf einen flachen Boden wurde das Fahrzeug gelegt und getestet. Das Fahrzeug fing an zu schwenken und konnte nicht absolut stehen bleiben, aber die Schwungamplitude war nicht groß. Das Verhalten war ordnungsmäßig und es konnte immer in einem festen Ort bleiben. Man kann sagen, die erste Aufgabe wurde erfüllt. Ein Video wurde aufgenommen, um das Verhalten des Fahrzeugs darzustellen.

Die zweite Aufgabe ist, das Fahrzeug über einen Bedienteil fernzusteuern. Mit den entsprechenden Steuersignalen fuhr das Fahrzeug jeweils nach vorne, nach hinten, nach links und nach rechts. Das Fahrzeug reagierte zwar langsam aber das Verhalten war stabil. Ein zweites Video wurde auch aufgenommen, um das noch genauer zu verdeutlichen. Zusammenfassende lässt sich sagen, dass die Ziele des Projekts grundsätzlich erledigt wurden.

3.3 Verbesserungsmöglichkeiten

Es gibt noch weitere Verbesserungsmöglichkeiten. Man kann zum Beispiel bei dem mechanischen Aufbau des Fahrzeugs eine andere Variante entwickeln, dessen Schwerpunkt anderes liegt. Die Lage des Schwerpunktes hat einen großen Einfluss auf die Stabilität. Mit einem hohen Schwerpunkt bekommt die Steuereinheit eine längere Regelzeit und somit steht mehr Zeit für den Algorithmus zur Verfügung. Die Stabilität wird daher verbessert. Aber dafür braucht man eine größere Resilienz und die Anforderungen an die Motoren werden erhöht. Mit einem niedrigen Schwerpunkt lässt sich das Fahrzeug einfacher, selbst ohne elektrische Regelung im Gleichgewicht zu halten. Wenn der Schwerpunkt ganz in der Nähe von der Motorachse liegt, bekommt das Fahrzeug eine größere Beschleunigung und kann sich schneller bewegen. Demzufolge ist der Bedarf an dem Drehmoment des Motors nicht so hoch. Bei einem zu niedrigen Schwerpunkt lässt sich die Steuerung aber schwierig durchzuführen.

Für erweiterte Funktionalität kann man zum Beispiel einige Infrarotsensoren im System nachträglich einbauen, um eine einfache Hinderniserkennung zu realisieren oder bei dem Bedienteil weitere Eingaben (Fahrgeschwindigkeit und Lenkwinkel) einfügen.

Referenz:

1. TB6612FNG Datenblatt

2. TM4C123GH6PM Datenblatt

3. Motordrehzahl mit PID-Steuerung https://wenku.baidu.com/view/d891ad1514791711cc791783.html

4. https://miaowlabs.com/wiki/Mwbalanced/Mathematical-Modeling.html

6. https://wolles-elektronikkiste.de/mpu6050-beschleunigungssensor-und-gyroskop

7. http://RobotControl.taobao.com